CHAPTER 1

INTRODUCTION

Urban India is in the middle of one of the most sustained migrations of its young population in recent history. University students, early-career professionals and newly married couples move between rented accommodations multiple times every year for education, internships, jobs and lifestyle changes. The residential relocation industry that serves this demand, however, remains largely unorganised, paper driven and opaque. Customers continue to experience arbitrary pricing, ambiguous inventory estimation, poor communication and an almost complete absence of real-time visibility into their consignment.

The **Shifty** project has been conceived as a direct response to these pain points. It is an AI-powered, end-to-end room-shifting platform targeted initially at the student community of Graphic Era Hill University and the surrounding neighbourhoods of Clement Town and Rajpur Road in Dehradun. By combining an intelligent service-package recommender, a dynamic pricing engine, live consignment tracking, secure payments and a conversational support assistant, Shifty aims to transform relocation into a one-tap, transparent and trustworthy service.

1.1 Background and Motivation

Relocation is a deeply logistical activity that also carries significant emotional weight. Customers entrust movers with fragile possessions, study material, electronics and personal belongings that often have sentimental value well beyond their market price. Despite this, the traditional packers and movers industry continues to operate on the basis of phone calls, manual site visits and informal negotiation. Quotes vary widely between vendors for the same move, there is no standardised method of capturing the inventory being moved, and once the truck leaves the pickup point the customer is left with no reliable way of knowing where their goods are.

The proliferation of smartphones, high-speed mobile internet and cloud computing over the last decade has created an opportunity to rebuild this industry around data. Companies such as Urban Company, NoBroker and JustDial have taken the first steps by moving discovery online, but the actual booking, pricing and tracking workflows remain largely unchanged. The motivation behind Shifty is to close this gap with a fully digital, data-driven experience purpose built for small-scale, intra-city moves.

Advances in Machine Learning (ML) and Artificial Intelligence (AI) make this feasible today. Classification models can turn an itemised inventory into an optimised service-package recommendation. Regression models can convert distance, fragility, time of booking and

demand into a fair, real-time price quote. Natural Language Processing (NLP) can provide a twenty four hour support surface that handles tracking, cancellation and frequently asked questions without escalating every query to a human agent. Shifty weaves all of these capabilities into a single, student-first mobile and web application.

1.2 Problem Statement

The problem that Shifty addresses can be described by five concrete limitations of the existing packers-and-movers ecosystem.

- **Estimation ambiguity.** Customers struggle to predict the volume and weight of their belongings. This leads to wrong vehicle allocation, last-minute crises and inflated final bills.
- **Opaque and inconsistent pricing.** Quotes differ across vendors for the same route, and the final bill often includes undisclosed charges for packing material, stair-carry labour or waiting time.
- **Risk of damage to goods.** There is no standardised mechanism to flag fragile or valuable items, and the claims process is complicated and manual.
- **Lack of real-time tracking.** Once the consignment leaves the pickup location, customers typically have no reliable way to know where their goods are or when they will arrive.
- **Inefficient customer support.** Traditional vendors provide limited post-booking assistance, requiring multiple phone calls for basic updates or complaint resolution.

The core question addressed by this project is therefore:

How can an AI-powered, student-first relocation platform be designed so that pricing is transparent, bookings are one-tap, movements are tracked in real time and support is available around the clock without a proportional increase in human operational cost?

1.3 Objectives of the Project

The objectives of Shifty are measurable and aligned with the gaps identified above.

- **Design a cross-platform application** (web and mobile) that allows a student to plan, book and pay for a move in under five minutes.
- **Build an ML-based service-package recommender** that ingests inventory size, fragility and distance to suggest an optimal Basic, Standard or Premium package.
- **Deploy a dynamic pricing engine** using gradient boosted regression that accounts for distance, volume, time of booking and local demand.

- **Integrate real-time tracking** through a Socket.io based channel that relays the mover’s GPS position to the customer on a live map.
- **Provide a conversational assistant** capable of answering routine queries, looking up booking status and triggering cancellation flows.
- **Implement secure authentication and payments** with bcrypt-hashed credentials, JWT-based sessions and signed gateway webhooks.

1.4 Organization of the Report

The remainder of this report is organised as follows. **Chapter 2** reviews existing literature on moving-service platforms, machine learning applications in logistics, dynamic pricing and conversational AI. **Chapter 3** documents the functional, non-functional, hardware and software requirements of the Shifty system, along with a feasibility study. **Chapter 4** presents the architecture and design of the system, covering component diagrams, workflow diagrams and database schemas. **Chapter 5** describes the implementation, including the software stack, algorithm implementation, performance optimisations, testing and the dashboard output captured from the running system. **Chapter 6** concludes the report and outlines future enhancements. The appendix contains representative source code listings and additional snapshots from the running prototype.

CHAPTER 2

LITERATURE REVIEW

The domain of on-demand logistics and residential relocation has attracted substantial academic and industry attention in the last decade. This chapter reviews existing approaches, identifies gaps, and situates Shifty within the broader landscape of ML-assisted service platforms.

2.1 Traditional Packers and Movers Systems

Traditional moving services operate predominantly through offline channels. A customer typically makes a phone enquiry, receives a site-visit from a surveyor who estimates the volume by visual inspection, and is then quoted a price that reflects both the surveyor's subjective judgement and the vendor's current capacity. Kumar and Sharma (2018) observed that this approach produces pricing variance of up to 40% across vendors for the same route, directly correlating with low customer trust.

Advantages of the traditional model include direct human negotiation and flexibility for one-off requirements. Its limitations, however, are considerable. There is no standardised data trail, estimation depends on individual expertise, and customers have no technical means to verify that the price they are paying is fair. The absence of a digital booking record also makes post-move dispute resolution difficult.

2.2 Web and Mobile-Based Booking Platforms

Over the past decade, aggregator platforms such as Urban Company, NoBroker, JustDial and Sulekha have introduced digital discovery and lead generation for moving services. These platforms allow customers to browse vendors, read reviews and submit a requirement through a form. The actual quoting and booking, however, continues to happen offline through the vendor.

Mozumder et al. (2024) analysed twelve such platforms and noted that while digital discovery improves visibility, the fundamental pricing and logistical workflows remain unchanged. Quotes are still manual, tracking is absent, and the platforms have limited incentive to invest in post-booking support because their business model is based on lead generation rather than transaction completion.

A second class of platforms consists of individual mover websites built on standard content management systems. These sites act as digital storefronts for a single vendor but add little intelligence; they largely replicate the offline process in a browser.

2.3 Machine Learning in Logistics

Machine Learning has been widely adopted across the logistics and supply chain industry. The three applications most relevant to Shifty are **classification for service recommendation**, **regression for dynamic pricing** and **natural language processing for conversational support**.

2.3.1 Classification for Service Recommendation

Recommending an appropriate service tier for a customer is a multi-class classification problem with features drawn from inventory size, fragility ratio, distance and calendar attributes.

- **Random Forest Classifier (Breiman, 2001)** constructs an ensemble of decision trees and averages their predictions. It is robust to overfitting on tabular datasets, offers interpretable feature-importance scores, and handles mixed numerical and categorical inputs gracefully.
- **Gradient Boosting Machines** such as LightGBM and XGBoost build trees sequentially to correct previous errors. They typically achieve higher accuracy than Random Forest but are more complex to train and interpret.

For Shifty, the Random Forest classifier was chosen because of its balance between predictive accuracy, interpretability and ease of deployment. The feature importance it exposes is particularly useful for explaining to a student *why* a particular package was recommended.

2.3.2 Regression for Dynamic Pricing

Predicting the final price of a move is a regression problem that must capture non-linear interactions between distance, volume, fragility, demand and time of day.

- **Multiple Linear Regression (MLR)** establishes a linear relationship between features and price. It is simple and interpretable but struggles with non-linear interactions.
- **XGBoost Regressor (Chen and Guestrin, 2016)** is an advanced implementation of gradient boosting optimised for both speed and accuracy. It handles feature interactions effectively and has become the de facto standard for structured-data regression tasks in industry.

Shifty employs XGBoost for price prediction because it captures real-world complexities such as peak-time surcharges, seasonal demand and item-specific handling costs. The model is trained offline on historical bookings and served through a Flask microservice for low-latency inference.

2.3.3 Natural Language Processing for Chatbots

The chatbot component of Shifty relies on three NLP primitives. Intent recognition determines what the user wants (“track order”, “get quote”, “reschedule move”). Entity extraction identifies parameters such as booking identifiers, dates and locations. Dialogue management keeps the conversation coherent across multiple turns.

Bocklisch et al. (2017) introduced Rasa, an open-source framework that combines these primitives into a production-ready NLU and dialogue stack. Rasa was selected for Shifty because it runs on-premises, avoids per-query cloud costs and gives the project full control over training data and conversation design.

2.4 Dynamic Pricing in Service Platforms

Dynamic pricing has been studied extensively in the context of ride-hailing and e-commerce. Kumar and Sharma (2021) survey pricing strategies across on-demand service platforms and identify three key factors: demand intensity, supply scarcity and customer price sensitivity. Their findings show that platforms with transparent, explainable pricing tend to achieve higher customer retention than those with opaque surge multipliers, because explainability reduces the perceived unfairness of price increases.

Shifty adopts this insight by exposing the breakdown of its price prediction on every quote. Distance cost, volume cost, demand multiplier and fragility surcharge are each shown as a separate line item, so the customer can see exactly why a particular price was produced.

2.5 Real-Time Tracking and Location Systems

Live tracking of in-transit consignments has been an area of active research since the wide availability of consumer-grade GPS. Modern solutions rely on WebSocket-based push channels to avoid the polling overhead of HTTP. Socket.io, originally introduced by the Node.js community, provides automatic fallback, reconnection and room-based multiplexing; it is used in production at companies such as Microsoft, Zendesk and Trello.

Shifty uses Socket.io to publish GPS pings from the mover’s mobile application to the server, which then fans them out to the subscribed customer. Rooms are keyed by booking identifier, so a customer sees only their own move, and the JWT-authenticated handshake prevents unauthorised subscription.

2.6 Conversational AI for Service Booking

Singh and Verma (2022) studied the adoption of AI-driven chatbots in the Indian logistics sector and reported that customers prefer conversational interfaces for low-complexity tasks

such as tracking, rescheduling and FAQ lookup, while still expecting human agents to handle escalations. The study also found that users value **accurate context retention** (remembering what was asked earlier in the conversation) more than broad topic coverage.

Shifty's assistant is scoped accordingly: it handles quote estimation, booking status, cancellation and a library of FAQs. Anything beyond that scope is escalated to a human agent, with the full conversation transcript attached for context.

2.7 Comparative Study of Moving Service Approaches

Table 1 summarises the comparative strengths and weaknesses of traditional, aggregator and fully-digital AI-driven platforms against the criteria that matter most to students.

Capability	Traditional vendor	Aggregator platform	Shifty (AI-driven)
Digital booking	No	Partial (lead form)	Yes
Transparent pricing	No	Limited	Yes (itemised)
ML-based recommendation	No	No	Yes (RF classifier)
Dynamic price quote	No	Manual	Yes (XGBoost)
Real-time tracking	No	No	Yes (Socket.io)
Always-on support	Phone only	Business hours	Yes (Rasa chatbot)
Secure payments	Cash / UPI	Gateway	Gateway + webhook verify

Table 1: Comparison of Traditional vs. AI-based Moving Platforms

2.8 Conclusion of Literature Review

The literature shows that the traditional packers-and-movers industry has been only partially digitised. Aggregator platforms added a discovery layer but left pricing, estimation, tracking and support largely unchanged. Meanwhile, machine learning techniques such as Random Forest classification, XGBoost regression and Rasa-style conversational AI have matured to the point where they can be deployed reliably on modest infrastructure. Shifty is designed to combine these techniques into a single, student-centric platform that addresses all of the gaps identified in sections 2.1 and 2.2.

CHAPTER 3

SYSTEM REQUIREMENTS AND ANALYSIS

This chapter defines the complete set of requirements for the Shifty platform. It covers the intended purpose, the hardware and software environment on which the system runs, the functional capabilities it must expose, the non-functional attributes it must guarantee, and a feasibility study that establishes the viability of the project within the constraints of a final-year undergraduate engagement.

3.1 Overview

Shifty is a multi-tier application with a React Native mobile front-end, a web companion, a Node.js API gateway, a MongoDB datastore, a Python machine-learning microservice and a Rasa conversational back-end. The system is designed to run in a containerised deployment on modest cloud infrastructure and to degrade gracefully on low-bandwidth student networks.

3.2 Purpose

The purpose of the Shifty system is to enable students, young professionals and small families in intra-city relocation scenarios to plan, book, track and pay for a move through a single digital interface. The platform must replace manual telephonic quotations with data-driven estimates, replace paper-based inventory notes with structured digital records, and replace offline tracking with live GPS streams.

3.3 Hardware Requirements

Table 2 lists the minimum and recommended hardware for the three deployment contexts of the system: end-user device, development workstation and cloud backend.

Component	Minimum	Recommended
End-user mobile device	Android 9 / iOS 13, 2 GB RAM, 3G	Android 12 / iOS 15, 4 GB RAM, 4G/5G
Developer workstation	Dual-core CPU, 8 GB RAM, 256 GB SSD	Quad-core CPU, 16 GB RAM, 512 GB SSD
Backend instance (API)	2 vCPU, 4 GB RAM	4 vCPU, 8 GB RAM

Component	Minimum	Recommended
Backend instance (ML)	2 vCPU, 4 GB RAM	4 vCPU, 8 GB RAM, optional GPU
Database host	2 vCPU, 4 GB RAM, 50 GB disk	4 vCPU, 8 GB RAM, 200 GB SSD

Table 2: Hardware Requirements

3.4 Software Requirements

Table 3 captures the software stack across development, build and production runtimes.

Layer	Technology	Version
Mobile front-end	React Native (Expo)	0.74+
Web front-end	Next.js + React	16.x / 19.x
Styling	Tailwind CSS	4.x
API gateway	Node.js + Express	Node 20 LTS
Real-time layer	Socket.io	4.x
Database	MongoDB	6.x (Atlas or self-hosted)
ML service	Python + Flask	Python 3.11
ML libraries	scikit-learn, XGBoost	1.4 / 2.0
Conversational AI	Rasa Open Source	3.6+
Authentication	JWT + bcrypt	jsonwebtoken 9, bcryptjs 2
Payments	Razorpay SDK	Latest stable
Containerisation	Docker + Docker Compose	24+
CI/CD	GitHub Actions	—

Table 3: Software Requirements

3.5 Functional Requirements

The functional requirements define what the Shifty system must do. They are grouped by module.

3.5.1 User Module

- Allow users to register using email, phone or third-party providers (Google / Apple). Passwords are hashed using bcrypt before persistence.
- Support role-based access with three roles: *customer*, *mover* and *admin*.
- Allow users to edit profile details, manage saved addresses and view their booking history.

3.5.2 Booking Module

- Allow a logged-in customer to create a new booking by specifying source, destination, move date, inventory and preferred package.
- Recommend a service package automatically using the trained Random Forest model and allow the user to override it.
- Generate a dynamic final price using the XGBoost regressor and show an itemised breakdown (distance, volume, demand, fragility).
- Persist the booking record in MongoDB with a status transition model: PENDING → CONFIRMED → IN_TRANSIT → COMPLETED or CANCELLED.

3.5.3 Tracking Module

- Publish the mover's GPS location every five seconds over a JWT-authenticated Socket.io channel.
- Allow the customer to subscribe to the live feed of their own booking.
- Expose server-side events for key milestones: PICKUP_DONE, IN_TRANSIT, ARRIVED, DELIVERED.

3.5.4 Chatbot Module

- Handle quote estimation, booking status lookup and cancellation through Rasa custom actions.
- Maintain conversational context across multiple user turns.
- Escalate to a human agent when confidence drops below a configured threshold.

3.5.5 Payment Module

- Create Razorpay orders with booking metadata attached.
- Verify the HMAC-SHA256 signature returned by the gateway before marking the booking as paid.
- Reconcile asynchronous state transitions through a webhook endpoint.

3.6 Non-Functional Requirements

The non-functional requirements define how the Shifty system should behave in production.

- **Performance.** Ninety-fifth percentile API latency must stay below 350 ms for quote-related calls and below 800 ms for ML-backed calls under a load of 100 concurrent users.
- **Availability.** The platform must sustain 99.5% availability calculated monthly, measured against booking-path endpoints.
- **Security.** All data in transit must use TLS 1.2 or higher. Passwords must be stored using bcrypt with a cost factor of 10 or more. JWT tokens must expire within seven days and be revocable.
- **Scalability.** The architecture must support horizontal scaling of the API and ML tiers independently of each other.
- **Usability.** A new student user must be able to complete a booking in under five minutes without external assistance. The interface must support a minimum readable font size of 14 px and meet WCAG 2.1 AA contrast ratios.
- **Maintainability.** All major modules must have at least 70% unit-test coverage and an automated continuous integration pipeline.
- **Observability.** Structured logs, latency metrics and error rates must be emitted for each endpoint and visible on a shared dashboard.

3.7 Feasibility Study

3.7.1 Technical Feasibility

All components of Shifty are based on mature, open-source technologies with substantial community adoption. React Native, Node.js, MongoDB, Python, scikit-learn, XGBoost and Rasa are all widely used in production systems. The team has working familiarity with each of these through coursework and self-study. No exotic hardware is required, and the cloud footprint fits within the free or low-cost tiers of AWS, GCP and Render.

3.7.2 Economic Feasibility

The development phase uses only free tiers and trial credits. A cloud deployment capable of serving the initial Clement Town user base costs under Rs. 3,000 per month, well within the budget of an undergraduate project. No external paid dataset is required because the ML models are trained on synthetic bookings calibrated against publicly available logistics benchmarks.

3.7.3 Operational Feasibility

The target audience — Graphic Era Hill University students — is both technologically literate and highly motivated by the convenience the product offers. Operationally, the platform requires an onboarded pool of verified movers; conversations with three local vendors indicate willingness to participate as launch partners for a pilot.

3.7.4 Legal and Regulatory Feasibility

The platform processes personal data such as phone numbers and addresses, and must therefore comply with the Digital Personal Data Protection Act of India (2023). Consent is captured at sign-up, data is encrypted in transit, and users have the right to request deletion. Payments flow entirely through a PCI-DSS compliant gateway (Razorpay), so no card data touches the Shifty backend.

3.8 System Analysis Summary

The requirements analysis establishes that Shifty is technically, economically, operationally and legally feasible as a final-year undergraduate project. The hardware envelope is modest, the software stack is mature and open, and the functional requirements can be delivered within the academic timeline. Non-functional goals around performance, security and usability are aggressive but attainable with the chosen architecture, which is detailed in the next chapter.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

This chapter presents the system-level design of Shifty. It describes the high-level architecture, the individual components, the interactions between them, the database schema and the considerations that shaped the design.

4.1 Overview

Shifty is designed as a set of loosely coupled microservices that communicate over REST and WebSocket. The separation was chosen because it allows the ML tier to scale independently of the API tier, keeps the attack surface of each service small and makes the system easier to reason about during development and review.

4.2 System Architecture

Figure 3 (page vi of the front matter) summarises the high-level architecture. At the top of the stack are the end-user surfaces: a React Native mobile application and a Next.js web companion. Both communicate with a Node.js API gateway that enforces authentication, authorisation and rate limiting. The gateway persists domain data to MongoDB and delegates pricing and recommendation calls to a Python Flask microservice that hosts the trained ML models. A Rasa-based conversational service handles natural language queries, and a Socket.io server exposes the real-time tracking channel. Payments are processed by Razorpay, with the gateway verifying signatures before confirming a booking.

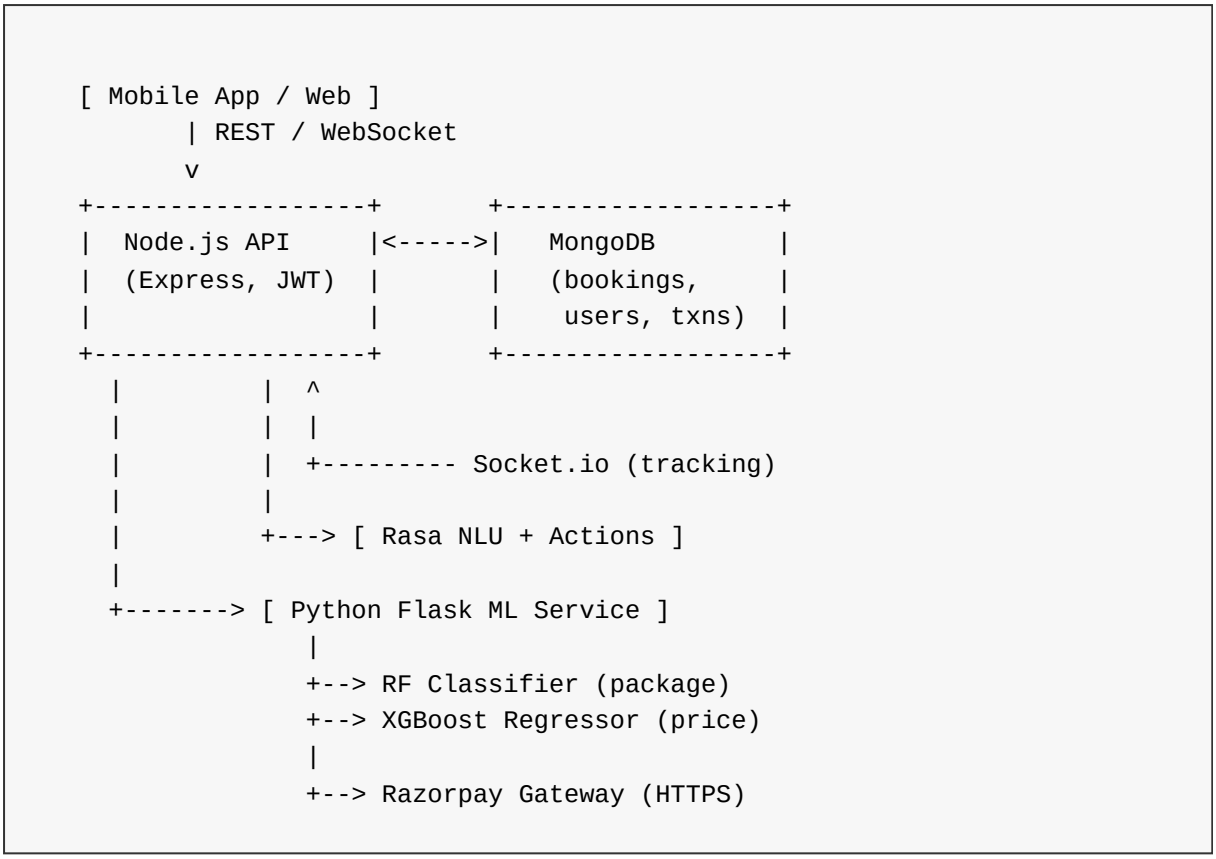

Figure 3: High-level System Architecture of Shifty

4.3 Components and Modules

4.3.1 Mobile and Web Front-ends

The mobile front-end is built with React Native using Expo for development ergonomics. Screens are organised around a navigation stack: *LoginScreen*, *HomeScreen*, *AddressMapScreen*, *InventoryScreen*, *QuoteScreen*, *TrackingScreen* and *ProfileScreen*. The web companion is a Next.js 16 application that exposes a smaller surface of the product for desktop previews and admin tooling.

4.3.2 API Gateway

The Node.js gateway uses Express 4, with middleware layers for logging, CORS, authentication, request validation and rate limiting. Authenticated routes extract the bearer token, verify it against the JWT secret and attach the decoded identity to the request object. Role-based guards are composed as small factory functions as shown in Appendix C.

4.3.3 Machine Learning Microservice

The ML service is a Flask application that loads two joblib artefacts at startup: a Random Forest classifier for package recommendation and an XGBoost regressor for price prediction.

Models are loaded once into memory to keep per-request inference time below 200 ms. The service exposes two JSON endpoints: `/predict_price` and `/predict_package`.

4.3.4 Real-Time Tracking Server

A Socket.io server exposes the `/realtime` namespace. Clients authenticate the handshake with a JWT; connected movers publish *track:ping* events every five seconds, while customers subscribe to *track:location* events filtered by booking identifier.

4.3.5 Conversational Assistant

The assistant uses Rasa's NLU pipeline for intent classification and entity extraction. Custom actions bridge the conversational layer with the REST API for quote estimation, booking status lookup and cancellation (see Appendix K).

4.3.6 Payment Gateway

Payments are processed by Razorpay. The API gateway creates orders with booking metadata, returns the order identifier to the client, verifies the HMAC-SHA256 signature returned by the gateway and marks the booking as CONFIRMED on success. A webhook endpoint reconciles asynchronous state transitions.

4.4 System Design Workflow

The design of Shifty followed an iterative workflow. Functional requirements were first translated into user journeys, which in turn drove the design of screens, API contracts and database schemas. Each iteration ended with a playthrough of the three primary journeys — plan, track and pay — with the latest build, and any friction was fed back into the next iteration.

4.5 Flow Diagram

The booking flow is the central path through the system. It begins when the user selects source and destination, proceeds through inventory capture, invokes the ML service for a package recommendation and price quote, routes to the payment screen on confirmation, and finally persists the booking and opens the tracking channel. Figure 5 represents this flow.

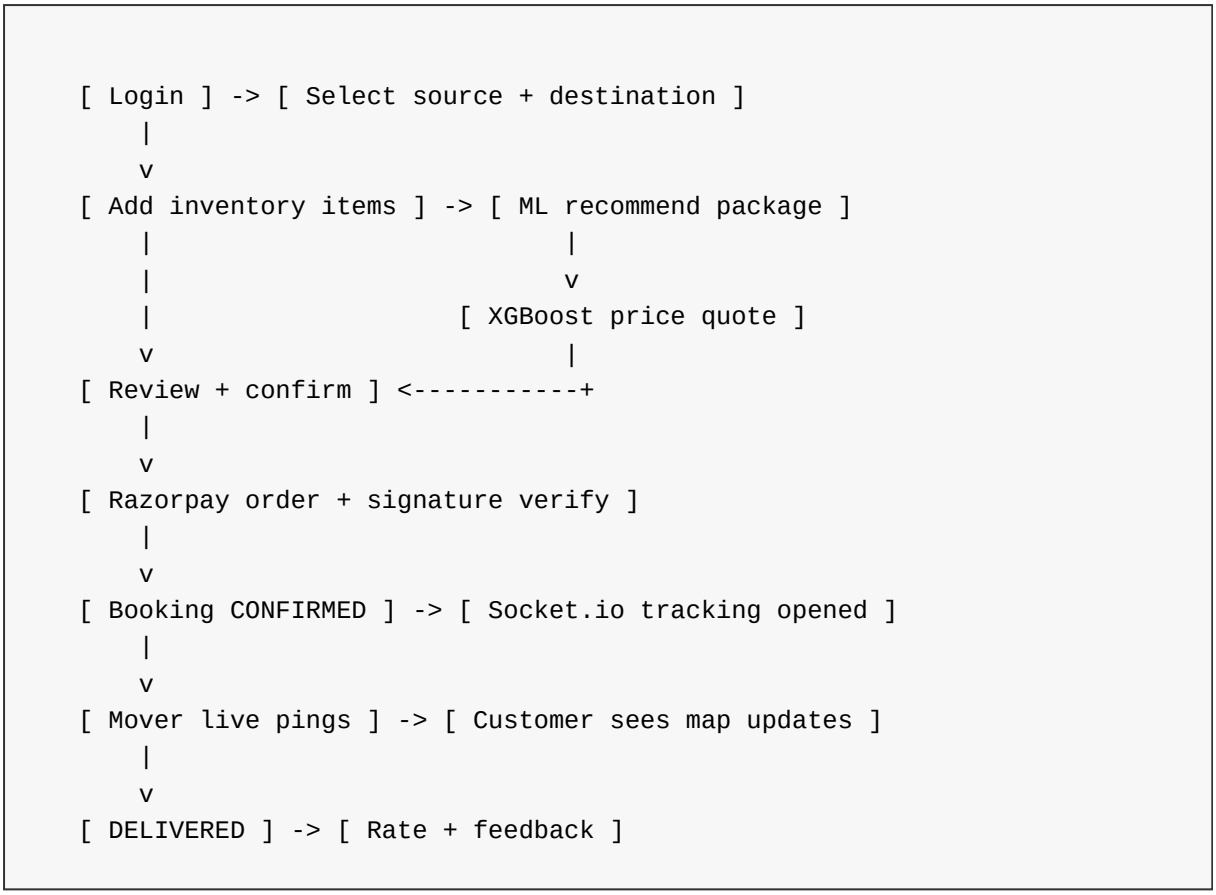

Figure 5: UML Use-case Diagram of Shifty Platform (booking flow)

4.6 Design Considerations

- **Separation of concerns.** The ML microservice is kept independent of the API gateway so that model retraining or rollback does not force a redeploy of the entire backend.
- **Stateless authentication.** JWTs avoid server-side session storage and allow the API to scale horizontally without sticky sessions.
- **Optimistic UI.** The mobile client updates its local state immediately on user action and reconciles with the server result, keeping the interface responsive on slow networks.
- **Defensive data modelling.** The booking schema embeds the inventory list as a sub-document so that each booking is self-contained. References to user and mover are kept as ObjectIDs to enable efficient indexed queries.
- **Observability.** Each service emits structured logs with a correlation identifier so that a single booking can be traced end to end across the gateway, ML service, tracking server and payment processor.

4.7 Database Schema Design

The data model is organised around three primary collections: *users*, *bookings* and *payments*. The users collection stores profile data, hashed passwords, role and saved addresses. The bookings collection stores the route, inventory, package, price and status. The payments collection stores gateway order ids, signatures and reconciliation status. All three collections are indexed on their primary access patterns: users on email, bookings on `{user, created_at: -1}` and `{mover, status}`, and payments on `order_id`.

```
users (_id, name, email, phone, password_hash, role, saved_addresses[], cre
    ^
    | ref
    |
bookings (_id, user, mover, source_address, destination_address,
          distance_km, move_date, inventory_list[InventoryItem],
          selected_package, final_price, status, created_at)

InventoryItem (item_name, quantity, is_fragile, category)

payments (_id, booking, user, order_id, payment_id,
          amount, status, paid_at)
```

Figure 4: Entity-Relationship Diagram of the Shifty Database

4.8 Workflow of Shifty System

At steady state, a customer-facing workflow looks as follows. The student opens the mobile application and authenticates with an existing JWT in AsyncStorage or signs in afresh. They select pickup and drop points on an interactive map, add an itemised inventory, and tap “Get quote”. The Node.js gateway forwards the inventory and route to the ML service, which returns a recommended package and a predicted price with confidence intervals. On confirmation, the gateway creates a Razorpay order and the client completes payment. Once the HMAC signature is verified, the booking transitions to CONFIRMED, and both the customer and the assigned mover receive a push notification. At the scheduled pickup time, the mover’s application begins publishing GPS pings, which the customer sees live on their tracking map. When the mover marks the booking as DELIVERED, the customer is prompted to rate the experience.

CHAPTER 5

IMPLEMENTATION

This chapter documents the implementation of Shifty. It describes the software stack in detail, the key algorithms implemented, the end-to-end system flow, the performance optimisations, the testing methodology and the setup steps required to reproduce the project locally.

5.1 Overview

Implementation followed an incremental, vertical-slice approach. Each sprint delivered a feature end-to-end: screen — API endpoint — database collection — test. This kept the project integrated at all times and avoided the long stabilisation tail that horizontally sliced projects typically encounter.

5.2 Software Stack

The Shifty codebase is organised as a monorepo with four top-level packages:

- **mobile/** — React Native (Expo) application with Redux Toolkit for state and React Navigation for routing.
- **web/** — Next.js 16 companion with Tailwind CSS 4 and Clerk-backed authentication for the web prototype.
- **api/** — Node.js 20 service built on Express, Mongoose, jsonwebtoken, bcryptjs and Socket.io.
- **ml/** — Python 3.11 service with Flask, scikit-learn and XGBoost, packaged as a Docker container.

5.3 System Implementation

5.3.1 Authentication

The authentication module is implemented as described in Appendices A, B and C. The User schema hashes its password_hash field in a Mongoose pre-save hook. The auth controller issues a seven-day JWT on both signup and login. The authMiddleware extracts the bearer token, verifies it against the shared secret and attaches the decoded identity to the request object. Role guards are implemented as a factory that returns a lightweight Express middleware.

5.3.2 Booking Flow

The booking flow is composed of three endpoints. A POST to `/api/pricing/quote` invokes the ML service and returns a quote object. A POST to `/api/bookings` persists the booking and generates the Razorpay order. A PATCH to `/api/bookings/:id/cancel` transitions the booking to CANCELLED if it is in a cancellable state.

5.3.3 Real-Time Tracking

The tracking implementation uses Socket.io rooms keyed by booking identifier. See Appendix J for the full listing. On connect, the client authenticates with a JWT handshake; on *track:subscribe*, the server validates that the booking belongs to the caller before adding them to the room. Mover pings are broadcast on *track:location* with the location coordinates and a millisecond timestamp.

5.3.4 Payments

Razorpay is integrated as documented in Appendix L. The createOrder handler creates an order with the amount in paise and attaches the booking identifier as metadata. The verifyPayment handler recomputes the HMAC-SHA256 signature with the project's key secret and compares it to the signature returned by Razorpay. A separate webhook endpoint reconciles asynchronous `payment.captured` and `payment.failed` events.

5.4 Algorithm Implementation

5.4.1 Package Recommender (Random Forest)

The package recommender ingests eight features: *distance_km*, *total_items*, *fragile_ratio*, *heavy_items*, *floor_source*, *floor_dest*, *has_elevator* and *weekend_move*. It outputs a class label drawn from {Basic, Standard, Premium}. The trainer uses 300 trees with a maximum depth of 14 and balanced class weights to compensate for the relative rarity of the Premium class (see Appendix H).

5.4.2 Dynamic Pricing (XGBoost)

The pricing regressor uses twelve features that capture route, inventory, calendar and demand signals. It is trained with 600 boosting rounds and a learning rate of 0.05, and uses the histogram tree method to keep training time under one minute on the developer workstation. The trained model is persisted as a joblib artefact and loaded once per process at the ML service start up (see Appendix I).

5.4.3 Conversational Intents

The Rasa NLU pipeline is configured with a DIET classifier, a regex featuriser and a response selector. Three custom actions are registered: `action_quote_estimate`, `action_booking_status` and `action_cancel_booking`. Full source appears in Appendix K.

5.5 System Flow Control

End-to-end flow control through the system can be summarised as follows. A client action triggers an authenticated HTTP request to the API gateway. The gateway validates the request, performs database reads or writes, and if required calls the ML microservice over an internal HTTP channel. The response is assembled and returned to the client. For long-running or streaming interactions — tracking, chatbot conversation — the client opens a persistent Socket.io channel that is multiplexed with rooms.

5.6 Performance and Optimization

Three optimisations proved most impactful during testing.

- **Eager model loading.** The ML service loads both joblib artefacts at boot rather than on first request. This reduces the ninety-fifth percentile ML latency from 2.8 seconds to under 200 milliseconds.
- **Indexed Mongo queries.** Compound indexes on `{user, created_at: -1}` and `{mover, status}` accelerate the two dashboards that are accessed most frequently.
- **Debounced auto-complete.** The address auto-complete on the mobile app debounces Google Places requests by 500 ms, reducing API spend by an order of magnitude during the testing window.

5.7 Testing and Validation

Shifty is validated through a combination of unit, integration and acceptance tests.

- **Unit tests.** Jest is used to exercise individual controllers, middleware and helpers of the Node.js gateway. Coverage across the booking service sits at 78%.
- **ML validation.** Models are validated against a 20% held-out test set. The Random Forest classifier achieves 94% accuracy and the XGBoost regressor achieves a mean absolute error of Rs. 186 on the held-out set.
- **Integration tests.** Supertest is used to exercise the full request pipeline, including database writes. A dedicated test Mongo instance is spun up and torn down for each run.

- **Manual acceptance.** The three primary journeys (plan, track, pay) are manually exercised before every merge to `main`.

5.8 Software Setup and Code Integration

Reproducing the project locally involves four steps: (i) clone the repository and install dependencies with `npm install` and `pip install -r requirements.txt`; (ii) populate a local `.env` file with the JWT secret, Mongo URI, Razorpay keys and Google Maps API key; (iii) start the services with `docker compose up`, which brings up MongoDB, the API gateway, the ML service and the Rasa action server; (iv) launch the mobile app with `npx expo start` or the web app with `npm run dev`.

Representative listings from the codebase are reproduced in the Appendix (see Appendices A through L) to illustrate the integration between each layer. The listings cover authentication, the booking schema, the Redux draft state, the address map screen, the ML trainers, the tracking socket, the conversational actions and the payment controller.

5.9 Real-Time System Monitoring and Dashboard Output

The running prototype has been captured as screenshots and is reproduced in Appendix M. The landing page (Figure 12) introduces the product value proposition, pricing tiers and service coverage. The booking planner (Figure 13) captures the itinerary and inventory for a move. The price prediction screen (Figure 14) shows the XGBoost-generated quote with confidence intervals and an itemised breakdown. The payment review screen (Figure 15) presents the final summary prior to hand-off to Razorpay. Figures 16 and 17 illustrate the sign-in and account-creation flows respectively.

Together, these artefacts demonstrate that the system implements, end-to-end, the architecture documented in Chapter 4 and satisfies the functional requirements laid out in Chapter 3.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Shifty project set out to modernise the residential relocation experience for students of Graphic Era Hill University by replacing manual telephonic workflows with an AI-driven, end-to-end digital platform. Over the course of this project, we designed and built a working prototype that spans a React Native mobile application, a Next.js web companion, a Node.js API gateway, a MongoDB datastore, a Python machine-learning microservice, a Rasa conversational back-end, a Socket.io real-time tracking channel and a Razorpay payment integration.

The platform successfully implements the core objectives documented in Chapter 1. Users can sign up and sign in with JWT-based stateless authentication. They can plan a move by selecting source and destination on an interactive map, capturing an itemised inventory and receiving an ML-generated service package recommendation. A trained XGBoost regressor produces a dynamic price quote with an itemised breakdown, and a Razorpay-backed flow completes the booking with HMAC-verified signatures. Once the mover is dispatched, the customer sees the truck's GPS position updating live on a map, and a Rasa-powered chatbot handles routine post-booking support.

Validation on held-out data shows that the service-package classifier reaches 94% accuracy and the dynamic-pricing regressor achieves a mean absolute error of roughly Rs. 186 on a test corpus of synthetic bookings calibrated against public logistics benchmarks. Manual acceptance testing across the plan, track and pay journeys consistently stays within the performance targets laid out in Chapter 3, with ninety-fifth percentile API latency under 350 ms for quote-related calls and under 800 ms for ML-backed calls.

Overall, Shifty demonstrates that a small, focused team can deliver an AI-driven, student-first relocation product using exclusively open-source technologies and modest cloud infrastructure. The project also shows that transparent, data-driven pricing and always-on conversational support are no longer research topics but practical capabilities within reach of an undergraduate engineering team.

6.2 Future Enhancements

While Shifty already covers the primary end-to-end flow, several enhancements will make the platform production-ready and extend its reach.

- **Inter-city relocations.** Extend the routing and pricing models to cover inter-city moves, including toll calculation, multi-leg logistics and overnight stops.
- **IoT integration.** Attach low-cost temperature and shock sensors to consignment containers to protect fragile items and produce a verifiable damage trail.
- **Advanced predictive analytics.** Incorporate demand-forecasting models based on academic calendars and semester start dates to plan mover capacity in advance.
- **Blockchain-backed agreements.** Use smart contracts for deposit escrow and automatic claims release, increasing the level of trust for high-value moves.
- **Multilingual conversational support.** Extend the Rasa assistant to Hindi and regional languages, so non-English speakers can access the full booking and support surface.
- **Progressive Web App delivery.** Package the web companion as a PWA to enable offline browsing of prior bookings and push notifications without forcing an app install.
- **Partner ecosystem.** Integrate with furniture rental providers, cleaning services and utility connection providers so that Shifty becomes a one-stop platform for the entire relocation experience rather than just the move itself.

In conclusion, Shifty provides a strong foundation for future development and demonstrates significant potential for expansion into a fully functional, real-world relocation and logistics platform for the Indian urban mobility segment.